NORTHSTAR TECHNOLOGIES
PVT. LTD.

DATABASE STANDARDS

Enterprise-wide guidance for responsible, secure and reliable data management

Document ID	ENG-GDE-009
Document Type	GUIDE
Access Scope	ALL_EMPLOYEES
Department	Not department-restricted
Version	1.0
Effective Date	04 November 2026
Document Owner	Engineering – Backend Engineering
Review Cycle	At least annually, and after material platform or security changes

Purpose. This guide establishes common principles for designing, using, protecting and maintaining databases and data stores across Northstar Technologies. It is intended to give every employee a shared understanding of sound data practices; implementation and production administration remain limited to authorized personnel.

Fictional demonstration document. This sample is for OnBoardIQ demonstration and RAG testing; it is not legal advice, a formally approved corporate policy, or a substitute for applicable contracts, laws, security requirements or platform-specific procedures.

1. Purpose and objectives

Databases and data platforms support business applications, analytics, employee services, operational workflows and AI-enabled systems. Consistent standards reduce avoidable defects and help protect the confidentiality, integrity and availability of company information.

The objectives are to promote understandable data models, reliable transactions, efficient queries, controlled access, recoverable systems, traceable changes and responsible data handling.

- Use data only for an authorized business purpose.
- Design data structures that are understandable, maintainable and proportionate to the application.
- Protect information throughout its lifecycle, from collection and storage to archival and deletion.
- Make database changes reviewable, testable and reversible where practical.

2. Scope and applicability

This guide applies enterprise-wide to employees, interns, contractors and approved service providers who design software, consume reports, handle data, operate systems, procure technology or make decisions that affect company data.

The ALL_EMPLOYEES access scope means the document may be retrieved by all authenticated employees. It does not grant every employee database credentials, administrative rights, unrestricted access to records or permission to change production data.

- Engineering teams must follow the technical standards when building or maintaining applications and data services.
- Data, analytics and AI/ML teams must apply relevant principles to datasets, pipelines, feature stores and vector databases.
- DevOps and IT Operations must apply the operational controls for hosting, monitoring, backup and recovery.
- All employees must follow data classification, acceptable-use and reporting expectations when viewing or handling information.

3. Core principles

- **Least privilege:** grant only the minimum access required for a person's role and current task.
- **Privacy by design:** avoid collecting or retaining personal or confidential data unless there is a documented business need.
- **Integrity:** enforce valid relationships, constraints and validation rules as close to the data layer as practical.
- **Availability:** plan for backup, recovery, capacity and operational failure.
- **Traceability:** record material schema changes and privileged operations through approved change and audit mechanisms.
- **Consistency:** use shared naming, modelling and migration conventions within a product or platform.

4. Data modelling and schema design

A schema should represent business concepts clearly and avoid unnecessary duplication. Choose relational, document, key-value, analytical or vector storage based on access patterns, consistency needs, scale and operational capability—not simply because a technology is fashionable.

- Define entities, attributes, relationships and ownership before implementation.
- Use normalization to reduce unintended duplication in transactional systems. Denormalize only when a measured access or performance requirement justifies it.

- Define primary keys for records and foreign keys where referential integrity is required and supported.
- Use NOT NULL, UNIQUE, CHECK and other constraints to enforce important invariants at the database layer.
- Choose data types that match the meaning and expected range of values. Avoid storing dates, numbers or structured values as arbitrary text.
- Represent timestamps consistently; store instants in UTC where possible and convert for display using the relevant business time zone.
- Document intentional denormalization, derived fields, soft-delete behaviour and data-retention assumptions.

5. Naming conventions

- Use clear, descriptive, stable names. Avoid unexplained abbreviations and reserved keywords.
- Follow one convention consistently within a service; a recommended relational convention is lowercase snake_case for schemas, tables and columns.
- Use singular or plural table names consistently across a project rather than mixing styles.
- Name primary and foreign-key constraints and indexes predictably, for example pk_employee, fk_order_customer and idx_employee_email.
- Use names that describe business meaning rather than implementation details that may change.
- Do not encode secrets, personal information or confidential values in object names.

6. Keys, relationships and data integrity

- Select stable primary keys. Use generated identifiers where they reduce coupling to mutable business attributes.
- Add foreign keys for relationships that must remain valid; document any deliberate decision not to enforce them in the database.
- Define uniqueness at the correct business boundary, including composite uniqueness where appropriate.
- Specify deletion and update behaviour deliberately. Avoid cascading deletes unless the impact is understood and approved.
- Validate input at application boundaries and retain database constraints as a final integrity safeguard.
- Use transactions when a business operation requires multiple changes to succeed or fail together.

7. Indexing and query performance

Indexes can improve reads but consume storage and add write overhead. Add them based on observed query patterns and execution plans, not by default to every column.

- Index columns frequently used in selective filters, joins and ordering when supported by query-plan evidence.
- Consider composite-index column order in relation to actual predicates and sort patterns.
- Review slow-query logs and query plans before changing indexes or rewriting queries.
- Avoid SELECT * in production paths where only a subset of columns is required.
- Use pagination for large result sets and prefer stable cursor/keyset pagination for deep pages where appropriate.
- Set sensible query timeouts and guard against unbounded scans or accidental high-volume requests.
- Reassess unused, duplicate or oversized indexes during routine maintenance.

8. Transactions and concurrency

- Use transactions to preserve atomicity for related updates.

- Choose the weakest isolation level that safely meets the business requirement; understand the anomalies permitted by the selected database and isolation level.
- Keep transactions short. Do not hold locks while waiting for user input or unrelated network calls.
- Handle deadlocks and serialization failures with bounded, safe retry logic where appropriate.
- Use optimistic concurrency controls or explicit locking when concurrent updates could overwrite important changes.
- Make retryable operations idempotent where feasible, especially when invoked through queues or distributed services.

9. Migrations and schema changes

Every schema change that affects a shared or production database must be reviewed, tested and recorded through the team's approved change process.

- Use version-controlled migration scripts; do not rely on undocumented manual edits.
- Test migrations against representative data volumes and supported application versions.
- Prefer backward-compatible expand-and-contract changes for rolling deployments.
- Plan for locks, table rewrites, execution time, replication lag and rollback limitations.
- Back up or otherwise establish a recovery point before high-risk changes.
- Never edit production schema directly to bypass review or deployment controls.
- Record the migration owner, purpose, deployment window, validation steps and recovery plan.

10. Security and access management

- Use individual identities or managed service identities; shared human database accounts should be avoided.
- Separate read-only, application, migration and administrative privileges.
- Store credentials in an approved secrets manager or secure configuration system; never commit passwords, tokens or connection strings containing secrets to source control.
- Use encrypted connections and encryption at rest where supported and required by the data classification.
- Restrict network access to approved workloads and administrative paths.
- Rotate credentials according to the applicable security standard and immediately after suspected exposure.
- Do not copy production data to local machines, test systems or external services without explicit authorization and appropriate safeguards.
- Use masked, synthetic or de-identified data for development and testing whenever possible.

11. Backups, recovery and availability

- Database owners must define backup frequency, retention, recovery objectives and responsibility based on business criticality.
- Protect backups with access controls and encryption appropriate to the source data.
- Keep backup copies logically or physically separated from the primary failure domain where feasible.
- Monitor backup jobs and investigate failures; a configured backup that has not been verified is not sufficient evidence of recoverability.
- Test restoration periodically and record the outcome, duration and issues.
- Document dependencies and recovery order for applications that rely on multiple data stores.
- Do not make unapproved changes to backup retention or disaster-recovery settings.

12. Monitoring, audit and operations

- Monitor availability, connection saturation, storage growth, replication health, backup status, error rates and query latency.
- Alert on actionable thresholds and assign an owner for investigation.
- Log administrative and security-relevant events in accordance with the approved logging and retention approach.
- Avoid logging passwords, access tokens, full payment details or unnecessary personal information.
- Use controlled maintenance windows for disruptive operations and communicate material service impact.
- Document operational runbooks for recurring tasks and known failure modes.

13. Data lifecycle and responsible use

Data must be collected and used for legitimate, approved purposes. Teams should know what data they hold, why it is needed, who owns it and how long it should be retained.

- Collect the minimum data necessary for the stated purpose.
- Apply classification and handling requirements to datasets, exports, backups and derived data.
- Follow approved retention schedules and legal holds; do not independently delete records subject to a preservation requirement.
- Securely delete or anonymize data when an approved retention period ends and deletion is authorized.
- Treat exports, spreadsheets, cached copies and analytics extracts as data assets subject to the same protection principles.

14. Non-relational, analytical and vector databases

The same security, ownership, lifecycle and reliability principles apply to non-relational systems. The implementation details differ by platform.

- For document databases, validate document shape and version schema changes where practical.
- For caches and key-value stores, define expiry, invalidation and acceptable data-loss behaviour.
- For analytical warehouses, manage lineage, refresh schedules, access boundaries and sensitive columns.
- For vector databases, record source-document identifiers and metadata, enforce authorization before retrieval, and define re-indexing and deletion procedures.
- Do not assume that embeddings or derived representations are non-sensitive merely because they are not human-readable.

15. Testing and release readiness

- Test schema constraints, migrations, transaction boundaries and failure handling.
- Use representative synthetic or appropriately masked datasets for performance and integration testing.
- Validate query plans and performance against expected workload patterns before high-impact releases.
- Test backup restoration and recovery procedures for critical data services.
- Include authorization tests to verify that users and services cannot access data outside their permitted scope.
- Confirm observability, rollback or recovery steps, ownership and deployment communication before release.

16. Roles and responsibilities

All employees: handle company data responsibly, use approved systems, protect credentials and report suspected exposure or misuse.

Application and engineering teams: design schemas, implement constraints, write safe queries, maintain migrations and test data access controls.

Data and AI/ML teams: maintain dataset quality, lineage, retention and access boundaries across analytical and vector systems.

DevOps / IT Operations: operate approved infrastructure, monitor services, manage backups and support recovery under authorized procedures.

Database or service owners: approve access, document critical dependencies, review capacity and coordinate material changes.

Specific ownership and approval authority must be assigned through Northstar's actual organizational and service-ownership processes.

17. Exceptions and incident reporting

A deviation from these standards must have a documented business reason, scope, owner, risk assessment, compensating controls and expiry or review date. Approval must come from the appropriate system or security owner under the company's governance process.

Suspected unauthorized access, data loss, credential exposure, unexpected data modification or database compromise must be reported promptly through the approved incident-reporting channel. Preserve relevant evidence and do not attempt unapproved remediation that could destroy evidence or worsen impact.

18. Production database change checklist

- Is there a documented business reason and accountable owner?
- Has the change been reviewed by the appropriate technical owner?
- Have migration scripts and rollback or recovery limitations been assessed?
- Have performance, locking and data-volume impacts been tested?
- Are access permissions limited to the required identities?
- Is a verified recovery point available for a high-risk change?
- Are monitoring, validation and communication steps defined?
- Have authorization and data-integrity tests passed?

19. Glossary

- **Primary key:** a column or set of columns that uniquely identifies a row.
- **Foreign key:** a constraint that represents and validates a relationship to another table.
- **Index:** a data structure that can speed up selected lookups at a storage and write-maintenance cost.
- **Migration:** a version-controlled change to a database schema or its managed data.
- **Least privilege:** granting only the access necessary to perform an authorized task.
- **RPO:** recovery point objective, the maximum tolerable amount of data loss measured in time.
- **RTO:** recovery time objective, the target time to restore a service after disruption.

Document review and governance

The document owner should review this guide at least annually and whenever major database platforms, data classifications, security requirements or operating practices change. Updates should be versioned and communicated through the approved internal process.

End of document. This document is a fictional Northstar Technologies Pvt. Ltd. demonstration artifact created for the OnBoardIQ sample knowledge base. All company procedures and examples are illustrative.